Step-by-step explanations

Step-by-step explanations of exercises 0.2 (the closed form), 0.3, 1.1 to 1.5, 1.9 and 1.10. Each one ends with a check on real numbers and a note on why the result matters for algorithms. The finished proofs are in `solutions.tex`, and the toolkit of prerequisites is in `how-to-solve.tex`.

Contents

0.2: Why we multiply the sum by c **1**

0.3: Fibonacci growth **2**

1.1: Adding three digits **5**

1.2: Binary versus decimal length **6**

1.3: Depth of a d-ary tree **7**

1.4: $\log(n!) = \Theta(n \log n)$ **10**

1.5: The harmonic sum **11**

1.9: Sums and products mod N **13**

1.10: Moving to a divisor **14**

0.2: Why we multiply the sum by c

The solution states the closed form

$$S = 1 + c + c^2 + \cdots + c^n = \frac{c^{n+1} - 1}{c - 1} \qquad (c \neq 1).$$

It comes from multiplying S by c. That shifts every term one place to the right, so cS and S overlap in all but one term at each end:

$$\begin{aligned} S &= 1 + c + c^2 + \cdots + c^n \\ cS &= \quad c + c^2 + \cdots + c^n + c^{n+1} \end{aligned}$$

Subtract. Every term from c to c^n appears in both and cancels:

$$cS - S = c^{n+1} - 1.$$

Factor out S and divide:

$$S(c-1) = c^{n+1} - 1 \quad \Longrightarrow \quad S = \frac{c^{n+1} - 1}{c - 1}.$$

Why c and not some other number? In a geometric series each term is c times the one before. Multiplying by c is the only scaling that makes the shifted copy line up with the original so the terms cancel.

Why $c \neq 1$? The last step divides by $c - 1$. When $c = 1$ the subtraction gives $0 = 0$, which says nothing, so that case is handled on its own: $n + 1$ ones add up to $n + 1$.

0.3: Fibonacci growth

Step 0: What the problem asks

The Fibonacci numbers start with 0 and 1, and each new number is the sum of the two before it:

n	0	1	2	3	4	5	6	7	8	9	10
F_n	0	1	1	2	3	5	8	13	21	34	55

The exercise traps F_n between two exponential functions:

- (a) A floor: F_n is at least $2^{n/2}$.
- (b) A ceiling: F_n is at most 2^{cn} for some c smaller than 1.
- (c) The best floor possible: the biggest c for which the floor still holds.

Step 1: How induction works

Picture a row of dominoes. To prove that all of them fall you need two things:

1. **Base case:** the first domino falls.
2. **Inductive step:** whenever one domino falls, it knocks over the next one.

Each F_n depends on the *two* numbers before it, so each domino needs the two behind it to fall first. That means you have to push over two dominoes at the start. This is why every part of this exercise has **two base cases**.

Step 2: One useful fact about exponents

Subtracting from the exponent is the same as dividing:

$$2^{(n-1)/2} = 2^{n/2} \cdot 2^{-1/2} = \frac{2^{n/2}}{\sqrt{2}}, \qquad 2^{(n-2)/2} = 2^{n/2} \cdot 2^{-1} = \frac{2^{n/2}}{2}.$$

So "one step back" divides by $\sqrt{2}$, and "two steps back" divides by 2.

Part (a): Show $F_n \geq 2^{n/2}$ for $n \geq 6$

Base cases. Push over the first two dominoes, $n = 6$ and $n = 7$:

- $F_6 = 8$ and $2^{6/2} = 2^3 = 8$. Since $8 \geq 8$, this holds (with equality).
- $F_7 = 13$ and $2^{7/2} = 2^3\sqrt{2} \approx 11.31$. Since $13 \geq 11.31$, this holds too.

Inductive step. Assume the claim is true for the two numbers before n:

$$F_{n-1} \geq 2^{(n-1)/2} \qquad \text{and} \qquad F_{n-2} \geq 2^{(n-2)/2}.$$

Add them, as the Fibonacci rule says to, and use Step 2:

$$F_n = F_{n-1} + F_{n-2} \geq \frac{2^{n/2}}{\sqrt{2}} + \frac{2^{n/2}}{2} = 2^{n/2}\left(\frac{1}{\sqrt{2}} + \frac{1}{2}\right).$$

The number in the brackets is about $0.707 + 0.5 = 1.207$. That is bigger than 1, so multiplying by it can only make $2^{n/2}$ bigger. So $F_n \geq 2^{n/2}$. ✓

Why $n \geq 6$? For small n the claim is false: $F_5 = 5$ but $2^{5/2} \approx 5.66$. At $n = 6$ the Fibonacci numbers overtake $2^{n/2}$, and from then on the inductive step keeps them ahead.

> **WATCH OUT**
>
> Checking $n = 6, 7, 8, 9, 10$ and stopping there is five examples, not a proof. The inductive step is what covers every n.

Part (b): Find $c < 1$ with $F_n \leq 2^{cn}$

$F_n \leq 2^n$ is easy, but that uses $c = 1$, and we need c to be *less* than 1. So we need a base smaller than 2.

The trick: the golden ratio. Let

$$\varphi = \frac{1 + \sqrt{5}}{2} \approx 1.618.$$

This number has a special property:

$$\varphi^2 = \varphi + 1.$$

(Check: $1.618^2 \approx 2.618$ and $1.618+1 = 2.618$.) Compare it to the Fibonacci rule $F_n = F_{n-1}+F_{n-2}$. The equation has the same shape: the next thing is the sum of the two before. That is why φ fits Fibonacci so well.

Claim: $F_n \le \varphi^n$ for all $n \ge 0$.

Base cases: $F_0 = 0 \le \varphi^0 = 1$ and $F_1 = 1 \le \varphi \approx 1.618$. ✓

Inductive step. Assume $F_{n-1} \le \varphi^{n-1}$ and $F_{n-2} \le \varphi^{n-2}$. Then

$$F_n = F_{n-1} + F_{n-2} \le \varphi^{n-1} + \varphi^{n-2} = \varphi^{n-2}(\varphi + 1) = \varphi^{n-2} \cdot \varphi^2 = \varphi^n. \quad \checkmark$$

The algebra closes up exactly. That is the payoff of choosing φ.

Converting to the form 2^{cn}. Any positive number can be written as a power of 2 using a logarithm: $\varphi = 2^{\log_2 \varphi}$. So choose

$$c = \log_2 \varphi \approx 0.694.$$

Then $2^{cn} = \varphi^n$, and $c < 1$ because $\varphi < 2$.

Part (c): The largest c with $F_n = \Omega(2^{cn})$

$F_n = \Omega(2^{cn})$ says that F_n is at least some fixed constant times 2^{cn} once n is large enough. A constant in front is allowed, but the growth rate has to keep up.

The answer is $c = \log_2 \varphi$, the same c as in part (b). There are two things to show.

1. This c works. Prove $F_n \ge \varphi^{n-2}$ for $n \ge 2$, with the same induction as in (b) but the inequality reversed:

- Base cases: $F_2 = 1 = \varphi^0$ and $F_3 = 2 \ge \varphi \approx 1.618$.
- Step: $F_n \ge \varphi^{n-3} + \varphi^{n-4} = \varphi^{n-4}(\varphi + 1) = \varphi^{n-4} \cdot \varphi^2 = \varphi^{n-2}$.

Since $\varphi^{n-2} = \varphi^n/\varphi^2 \approx 0.38\,\varphi^n$, we get $F_n \ge 0.38 \cdot 2^{cn}$. The 0.38 is the constant in front, which Ω allows.

2. No bigger c works. Take any $c' > \log_2 \varphi$. Then the base $B = 2^{c'}$ is larger than φ. From part (b), $F_n \le \varphi^n$, so

$$\frac{F_n}{B^n} \le \left(\frac{\varphi}{B}\right)^n.$$

The fraction φ/B is less than 1, and a number less than 1 raised to bigger and bigger powers shrinks toward 0 (think 0.9^n). So F_n becomes a smaller and smaller fraction of B^n, and no fixed constant can make up the gap.

Summary

$$0.38\,\varphi^n \;\le\; F_n \;\le\; \varphi^n$$

Fibonacci grows like $\varphi^n \approx 1.618^n$. Part (a) gives a weaker floor, since $2^{n/2} = (\sqrt{2})^n \approx 1.414^n$.

Check: $F_{10} = 55$ and $\varphi^{10} \approx 122.99$, so $0.38 \times 122.99 \approx 47$. Since $47 \le 55 \le 123$, the bounds hold.

1.1: Adding three digits

The problem: in any base $b \geq 2$, the sum of three single-digit numbers has at most two digits.

Step 0: What "base b" means

In base 10 there are ten digits, 0 to 9. After 9 we run out and write "10", meaning one ten and zero ones. Base b works the same way with b digits:

- The digits are $0, 1, \ldots, b-1$, so the largest single digit is $b-1$.
- Writing "10" means b, and "100" means b^2.

Base	Digits	Largest digit	"100" means
10	0–9	9	100
2	0, 1	1	4
16	0–9, A–F	F (15)	256

Step 1: Try the worst case first

To show something is never too big, check the biggest possible case. If that one fits, every smaller case fits too. The biggest sum is three copies of the largest digit:

$$(b-1) + (b-1) + (b-1) = 3b - 3.$$

In base 10: $9 + 9 + 9 = 27$, which has two digits.

Step 2: How big can a two-digit number get?

The smallest three-digit number in base b is "100", which equals b^2. Every number below b^2 has at most two digits. So we need to show

$$3b - 3 < b^2.$$

Step 3: Do the algebra

Moving everything to one side gives $b^2 - 3b + 3 > 0$. It is tidier to prove the slightly stronger $b^2 - 3b + 2 \geq 0$: if that is at least 0, then one more is definitely bigger than 0. Factor it:

$$b^2 - 3b + 2 = (b-1)(b-2).$$

Step 4: Why the product is never negative

Since $b \geq 2$, the factor $b - 1 \geq 1$ is positive and the factor $b - 2 \geq 0$ is zero or positive. Their product is zero or positive. So $3b - 3 < b^2$: the biggest possible sum is less than "100" and fits in two digits.

Step 5: Check with examples

Base	Worst case	In base 10	In base b	Digits
10	$9+9+9$	27	27	2
2	$1+1+1$	3	11	2
16	F + F + F	45	2D	2

Base 2 is the tightest case: the sum 3 is 11_2, and 4 would already be 100_2. That matches the algebra, since $(b-1)(b-2) = 1 \times 0 = 0$ when $b = 2$.

WHY THIS MATTERS

When you add two long numbers by hand, each column adds two digits plus a carry, which is three single digits. This exercise shows the column total always fits in two digits: one to write down and a carry of at most 1. That holds in every base, which is why schoolbook addition works in binary just as well as in base 10.

1.2: Binary versus decimal length

The problem: show that a number written in binary is never more than 4 times as long as the same number in decimal, and find the approximate ratio for very large numbers.

Step 0: Look at some examples

Number	Binary	Decimal digits	Binary digits	Ratio
5	101	1	3	3
9	1001	1	4	**4**
99	1100011	2	7	3.5
1000	1111101000	4	10	2.5

The ratio stays at 4 or below, and 9 hits exactly 4.

Step 1: Counting digits with powers

The decimal numbers with exactly 3 digits are 100 to 999, that is $10^2 \le N < 10^3$. In general, N has d decimal digits exactly when $10^{d-1} \le N < 10^d$, and b bits exactly when $2^{b-1} \le N < 2^b$. Taking logs gives

$$\text{digits of } N \text{ in base } B = \lfloor \log_B N \rfloor + 1.$$

Check: $\log_{10} 999 \approx 2.9996$, which rounds down to 2, and $2 + 1 = 3$ digits.

Step 2: The key number, $\log_2 10 \approx 3.32$

One decimal digit can be any of 10 values. Three bits give 8 values (not enough) and four give 16 (more than enough). The exact number of bits needed for 10 values is the x with $2^x = 10$, which is $\log_2 10 \approx 3.32$. So binary should be about 3.32 times as long as decimal. The proof makes sure that rounding to whole digits never pushes the ratio past 4.

Write d for the number of decimal digits and b for the number of bits. We want $b \leq 4d$.

Step 3: Case 1, one decimal digit ($d = 1$)

The numbers are 1 to 9. The largest is $9 = 1001_2$, with 4 bits. So $b \leq 4 = 4d$.

Step 4: Case 2, two or more digits ($d \geq 2$)

A d-digit number is less than 10^d, so

$$b = \lfloor \log_2 N \rfloor + 1 \leq \log_2 N + 1 < \log_2(10^d) + 1 = d \log_2 10 + 1 < 3.33\, d + 1.$$

Since $d \geq 2$, we have $1 \leq d/2$, so the leftover 1 can be swapped for $d/2$:

$$b < 3.33\, d + 0.5\, d = 3.83\, d < 4d. \quad \checkmark$$

Why Case 1 is separate: when $d = 1$ the "+1" adds a whole extra digit on top of 3.32, giving up to 4.32, so the algebra alone does not prove the bound. Step 3 checks that case directly.

Step 5: The ratio for very large numbers

$$\frac{b}{d} = \frac{\lfloor \log_2 N \rfloor + 1}{\lfloor \log_{10} N \rfloor + 1}.$$

When N is huge, the rounding and the +1s are tiny compared to the logs, so what is left is

$$\frac{\log_2 N}{\log_{10} N} = \log_2 10 \approx 3.32$$

by the change-of-base rule $\log_2 N = \log_2 10 \cdot \log_{10} N$.

Example: $N = 10^{100}$ has 101 decimal digits and 333 bits, a ratio of about 3.30.

WHY THIS MATTERS

Running times are often measured by the number of digits in the input. Switching between binary and decimal only multiplies the length by a constant, about 3.32, and big-O ignores constant factors. So an $O(n^2)$ algorithm is $O(n^2)$ whether you count bits or decimal digits.

1.3: Depth of a d-ary tree

The problem: a d-ary tree is a rooted tree in which every node has at most d children. Show that a d-ary tree with n nodes has depth $\Omega(\log n / \log d)$, and find an exact formula for the smallest possible depth.

Step 0: The words

- **Root:** the one node at the top. Each node can have children below it.
- **d-ary:** each node has at most d children. A binary tree is the case $d = 2$.
- **Depth of a node:** the number of steps (edges) from the root. The root has depth 0.
- **Depth of the tree:** the depth of its deepest node, that is, how tall the tree is.

WHAT THE LETTERS MEAN

n	the total number of nodes in the tree, counting every node once, including the root
D	how tall the tree is (its depth)
d	the most children any one node can have

The exercise gives you n and asks how short a tree holding n nodes can be. n means something different in each exercise: in 1.2 it was the number itself, and in 0.3 it was the position in the Fibonacci sequence.

Step 1: The big idea

A short tree cannot hold many nodes, because each level can only be d times as wide as the one above it. So fitting n nodes forces the tree to be tall enough. Think of a building where each floor holds d times as many people as the floor above: a big crowd needs enough floors.

Step 2: How many nodes fit on each level?

A full binary tree has 1 node on level 0, 2 on level 1, 4 on level 2, and so on. In general each level holds at most d times as many nodes as the level above:

Level	0	1	2	3	$\cdots$	k
Most nodes	1	d	d^2	d^3	$\cdots$	d^k

Step 3: Total nodes in a tree of depth D

A tree of depth D has levels $0, 1, \ldots, D$. Adding up the most each level can hold gives the geometric sum from 0.2 with $c = d$:

$$n \le 1 + d + d^2 + \cdots + d^D = \frac{d^{D+1} - 1}{d - 1}.$$

Check ($d = 2$, $D = 2$): $1 + 2 + 4 = 7$ and $(2^3 - 1)/(2 - 1) = 7$.

Step 4: Simplify

Since $d \ge 2$, the bottom $d - 1$ is at least 1, and the top is less than d^{D+1}. So

$$n < d^{D+1}.$$

Step 5: Solve for D using a log

Logs undo powers: $\log_d(d^x) = x$. Taking $\log_d$ of both sides,

$$\log_d n < D + 1,$$

so $D > \log_d n - 1$. The depth has to grow at least as fast as $\log_d n$.

Step 6: Why the answer is written $\log n / \log d$

By the change-of-base rule $\log_d n = \log n / \log d$, so

$$D = \Omega\left(\frac{\log n}{\log d}\right).$$

Written this way, the role of d is visible: more children per node means a shorter tree.

Step 7: The exact smallest depth

To make the tree as short as possible, fill every level before starting the next. The smallest depth is then the smallest D whose full tree has room for n nodes:

$$\frac{d^{D+1} - 1}{d - 1} \geq n \iff d^{D+1} \geq 1 + n(d-1) \iff D + 1 \geq \log_d\big(1 + n(d-1)\big).$$

D has to be a whole number, so round up (the ceiling):

$$\boxed{D = \left\lceil \log_d\big(1 + n(d-1)\big) \right\rceil - 1}$$

Step 8: Check with examples

- $d = 2$, $n = 7$: $\log_2 8 = 3$, so $D = 2$. Depth 2 holds $1 + 2 + 4 = 7$ nodes exactly.
- $d = 2$, $n = 8$: $\log_2 9 \approx 3.17$, so $D = 4 - 1 = 3$. The three full levels are used up, so node 8 starts a new level.
- $d = 3$, $n = 13$: $\log_3 27 = 3$, so $D = 2$. Depth 2 holds $1 + 3 + 9 = 13$ nodes exactly.

Special case $d = 1$: the tree is a straight line, so its depth is exactly $n - 1$. The formula breaks because $\log_1$ does not exist, which is why the solution assumes $d \geq 2$.

WHY THIS MATTERS

Many algorithms can be drawn as trees: each decision splits into a few choices, and the depth is the number of steps taken. If there are n possible outcomes, some path needs at least about $\log_d n$ steps. That is why binary search takes $\log_2 n$ steps, and why no comparison-based method can find an item among n in fewer.

1.4: $\log(n!) = \Theta(n \log n)$

The hint: for the upper bound compare $n!$ with n^n; for the lower bound compare it with $(n/2)^{n/2}$. Here n is just a positive whole number.

Step 0: What the symbols mean

Factorial: $n!$ multiplies all the whole numbers from 1 to n, so $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$.

Θ: $f(n) = \Theta(g(n))$ means f and g grow at the same rate: once n is large, f stays between two fixed multiples of g. You prove it in two halves, an upper bound (O) and a lower bound (Ω).

Step 1: Why compare with simpler products?

Every factor of $n!$ is different, which makes it awkward. The trick is to trap it between two products where every factor is the same. Those are powers, and powers are easy to take logs of: $\log(a^k) = k \log a$.

Step 2: Upper bound, compare with n^n

$n!$ has n factors and every one is at most n. Replacing each with n can only make the product bigger:

$$n! = 1 \times 2 \times \cdots \times n \;\le\; \underbrace{n \times n \times \cdots \times n}_{n \text{ times}} = n^n.$$

Example: $5! = 120 \le 5^5 = 3125$. Taking logs (bigger numbers have bigger logs),

$$\log(n!) \le n \log n,$$

so $\log(n!) = O(n \log n)$. ✓

Step 3: Lower bound, compare with $(n/2)^{n/2}$

1. **Throw away the small half of the factors.** Every factor is at least 1, so removing factors can only make the product smaller.
2. **Each factor left is at least $n/2$.** Replace each one with $n/2$, which makes the product smaller again.

Example ($n = 8$): throw away $1 \times 2 \times 3$ and keep $4 \times 5 \times 6 \times 7 \times 8$. At least four of the kept factors are ≥ 4, so $8! \ge 4^4 = 256$, and indeed $8! = 40{,}320$.

In general at least $\lfloor n/2 \rfloor$ factors are $\ge n/2$, so

$$n! \ge \left(\frac{n}{2}\right)^{\lfloor n/2 \rfloor} \quad \Longrightarrow \quad \log(n!) \ge \left\lfloor \frac{n}{2} \right\rfloor (\log n - \log 2)\,.$$

Step 4: Why that is still $n \log n$-sized

Both pieces are only a constant factor smaller: $\lfloor n/2 \rfloor$ is about half of n, and $\log n - \log 2$ is about $\log n$ once n is large. Precisely (with $\log_2$), for $n \geq 4$ we have $\lfloor n/2 \rfloor \geq n/4$ and $\log n - 1 \geq \frac{1}{2} \log n$, so

$$\log(n!) \geq \frac{n}{4} \cdot \frac{\log n}{2} = \frac{1}{8} n \log n.$$

A fixed number times $n \log n$ is what $\Omega(n \log n)$ requires. ✓

Step 5: Put the halves together

$$\frac{1}{8} n \log n \;\leq\; \log(n!) \;\leq\; n \log n \qquad (n \geq 4),$$

so $\log(n!) = \Theta(n \log n)$.

Step 6: Check with numbers ($n = 8$, logs base 2)

Lower bound $4 \log_2 4 = 8$; actual $\log_2 40{,}320 \approx 15.3$; upper bound $8 \log_2 8 = 24$. And $8 \leq 15.3 \leq 24$.

WHY THIS MATTERS

This is why sorting cannot be done faster than $n \log n$ using comparisons. n items can be arranged in $n!$ orders, and a sorting algorithm has to tell them all apart. Each comparison is a yes/no question, so the algorithm's choices form a binary tree with at least $n!$ leaves. By exercise 1.3 that tree has depth at least about $\log_2(n!)$, and by this exercise that is $\Theta(n \log n)$. Merge sort reaches this bound.

1.5: The harmonic sum

The problem: show that

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \Theta(\log n).$$

The hint: for an upper bound, decrease each denominator to the next power of 2; for a lower bound, increase it to the next power of 2.

Step 0: Why this is surprising

The terms keep shrinking, so you might expect the total to level off, like $1 + \frac{1}{2} + \frac{1}{4} + \cdots$ in 0.2, which never passes 2. It does not. The harmonic sum grows forever, just very slowly:

n	1	2	4	8	100	1,000,000
H_n	1	1.5	2.08	2.72	5.19	14.39

Step 1: The strategy, group and round

There is no neat formula for H_n, so we trap it. Group the fractions into blocks that start at powers of 2:

$$\underbrace{1}_{\text{block 0}} + \underbrace{\tfrac{1}{2}+\tfrac{1}{3}}_{\text{block 1}} + \underbrace{\tfrac{1}{4}+\tfrac{1}{5}+\tfrac{1}{6}+\tfrac{1}{7}}_{\text{block 2}} + \underbrace{\tfrac{1}{8}+\cdots+\tfrac{1}{15}}_{\text{block 3}} + \cdots$$

Each block has twice as many terms as the one before, but each term is about half the size. So every block adds up to roughly the same amount.

Step 2: Upper bound, round the denominators down

A smaller denominator makes a bigger fraction, so rounding every denominator in a block down to the power of 2 at its start can only make the sum bigger:

$$\tfrac{1}{2}+\tfrac{1}{3} \le \tfrac{1}{2}+\tfrac{1}{2} = 1, \qquad \tfrac{1}{4}+\tfrac{1}{5}+\tfrac{1}{6}+\tfrac{1}{7} \le 4\times\tfrac{1}{4} = 1, \qquad \tfrac{1}{8}+\cdots+\tfrac{1}{15} \le 8\times\tfrac{1}{8} = 1.$$

Every block is at most 1. Block k starts at 2^k, so the blocks that contain any terms are $k = 0, 1, \ldots, m$ with $m = \lfloor \log_2 n \rfloor$. That is $m+1$ blocks (the last may be cut off at n, which only makes it smaller), so

$$H_n \le m + 1 \le \log_2 n + 1,$$

and $H_n = O(\log n)$. ✓

Step 3: Lower bound, round the denominators up

A bigger denominator makes a smaller fraction, so rounding up can only make the sum smaller. This time group so that each block ends at a power of 2:

$$1 + \tfrac{1}{2} + \left(\tfrac{1}{3}+\tfrac{1}{4}\right) + \left(\tfrac{1}{5}+\tfrac{1}{6}+\tfrac{1}{7}+\tfrac{1}{8}\right) + \cdots$$

Round each denominator up to the power of 2 at the end of its block:

$$\tfrac{1}{2} = \tfrac{1}{2}, \qquad \tfrac{1}{3}+\tfrac{1}{4} \ge 2\times\tfrac{1}{4} = \tfrac{1}{2}, \qquad \tfrac{1}{5}+\tfrac{1}{6}+\tfrac{1}{7}+\tfrac{1}{8} \ge 4\times\tfrac{1}{8} = \tfrac{1}{2}.$$

Every block is at least $\frac{1}{2}$. Since $n \ge 2^m$, the blocks ending at $2, 4, \ldots, 2^m$ are all complete. That is m blocks plus the 1 at the start:

$$H_n \ge 1 + \frac{m}{2}.$$

Rounding down loses less than 1, so $m > \log_2 n - 1$, and

$$H_n \ge 1 + \frac{\log_2 n - 1}{2} = \frac{\log_2 n + 1}{2}.$$

That is at least half of $\log_2 n$, so $H_n = \Omega(\log n)$. ✓

Step 4: Put the halves together

$$\frac{\log_2 n + 1}{2} \;\le\; H_n \;\le\; \log_2 n + 1,$$

so $H_n = \Theta(\log n)$.

Step 5: Check with numbers ($n = 8$, so $m = 3$)

$H_8 = 1 + 0.5 + 0.333 + 0.25 + 0.2 + 0.167 + 0.143 + 0.125 \approx 2.72$. The lower bound is $1 + \frac{3}{2} = 2.5$ and the upper bound is $3 + 1 = 4$, and $2.5 \leq 2.72 \leq 4$.

BONUS: WHY THE FULL SERIES NEVER STOPS GROWING

Step 3 shows every block adds at least $\frac{1}{2}$, and there are infinitely many blocks. Adding $\frac{1}{2}$ over and over without end has no limit, so $1 + \frac{1}{2} + \frac{1}{3} + \cdots$ is infinite even though its terms shrink to 0. In the geometric series from 0.2, each block would be half the size of the one before, so the total stays bounded.

WHY THIS MATTERS

Harmonic sums appear whenever an algorithm does $\frac{n}{1} + \frac{n}{2} + \frac{n}{3} + \cdots$ work, for example looping over every i and then over all multiples of i up to n. That total is $nH_n = \Theta(n \log n)$, not $\Theta(n^2)$. The harmonic sum also shows up in the average-case analysis of quicksort and in the coupon collector problem.

1.9: Sums and products mod N

The problem: suppose $x \equiv x' \pmod{N}$ and $y \equiv y' \pmod{N}$. Prove that

$$x + y \equiv x' + y' \pmod{N} \qquad \text{and} \qquad xy \equiv x'y' \pmod{N}.$$

In words: if you swap a number for another one with the same remainder, sums and products still come out with the same remainder.

SAYING IT ALOUD

x' is read "x prime" (some people say "x dash"). The mark only shows that it is a different number related to x; here it is "another number with the same remainder as x". It is not a derivative. The statement reads: "If x is congruent to x prime mod N, and y is congruent to y prime mod N, then x plus y is congruent to x prime plus y prime mod N."

Step 0: What "mod" means

On a clock, 10 o'clock plus 5 hours is 3 o'clock, not 15: a clock wraps around after 12. "Mod N" is clock arithmetic with N hours. Two numbers are congruent mod N if they leave the same remainder when divided by N:

$$17 \equiv 2 \pmod{5} \qquad \text{because } 17 = 3 \times 5 + 2 \text{ and } 2 = 0 \times 5 + 2.$$

Step 1: Turn "≡" into an equation

"Same remainder" is hard to do algebra with. Equivalently, $x \equiv x' \pmod{N}$ means x and x' differ by a multiple of N:

$$x - x' = aN \qquad \text{or} \qquad x = x' + aN$$

for some whole number a. Check: $17 - 2 = 15 = 3 \times 5$.

This is the most important step. Once the $\equiv$ signs are ordinary equations, both proofs are plain algebra. From what we are given,

$$x = x' + aN, \qquad y = y' + bN$$

for some whole numbers a and b. The goal in each part is to show the difference between the two sides is a whole number times N.

Step 2: Addition

$$x + y = (x' + aN) + (y' + bN) = (x' + y') + (a + b)N,$$

so

$$(x + y) - (x' + y') = (a + b)N.$$

$a + b$ is a whole number, so $x + y \equiv x' + y' \pmod{N}$. ✓

Step 3: Multiplication

$$xy = (x' + aN)(y' + bN) = x'y' + x'bN + aNy' + abN^2.$$

Every term except the first has a factor of N, so take it out:

$$xy - x'y' = N\big(x'b + y'a + abN\big).$$

The bracket is whole numbers multiplied and added, so it is a whole number, and $xy \equiv x'y' \pmod{N}$. ✓

Step 4: Check with numbers (mod 5)

Take $x = 17$, $x' = 2$ (both remainder 2) and $y = 9$, $y' = 4$ (both remainder 4).

- Sum: $17 + 9 = 26$ and $2 + 4 = 6$ both have remainder 1.
- Product: $17 \times 9 = 153$ and $2 \times 4 = 8$ both have remainder 3.

WHY THIS MATTERS

You can reduce mod N at any point in a calculation without changing the final answer. To find 7^{100} mod 13 you never need the 85-digit value of 7^{100}: multiply step by step and take the remainder after each step, so the numbers never exceed 13×13. This is how computers do the huge-number arithmetic in RSA encryption, later in Chapter 1.

1.10: Moving to a divisor

The problem: show that if $a \equiv b \pmod{N}$ and M divides N, then $a \equiv b \pmod{M}$.

Step 0: A clock example first

12 divides 24. If two times show the same reading on a 24-hour clock, they must also show the same reading on a 12-hour clock. The 12-hour clock tells you less (it cannot tell 1 a.m. from 1 p.m.), but it never disagrees with the 24-hour clock. That is the whole exercise.

Step 1: Translate everything into equations

"$a \equiv b \pmod{N}$" means $a - b = kN$ for some whole number k.

"M divides N" means $N = mM$ for some whole number m. (For example, 4 divides 12 because $12 = 3 \times 4$.)

Goal: show that $a - b$ is a multiple of M.

Step 2: Substitute

$$a - b = k(mM) = (km)M.$$

km is a whole number, so $a - b$ is a multiple of M, and $a \equiv b \pmod{M}$. ✓

In one sentence: a multiple of N is automatically a multiple of anything that divides N.

Step 3: Check with numbers

$26 \equiv 2 \pmod{12}$ because $26 - 2 = 24 = 2 \times 12$. Since 4 divides 12, also $24 = 6 \times 4$, so $26 \equiv 2 \pmod{4}$: both leave remainder 2.

WATCH OUT: IT DOES NOT WORK BACKWARDS

$5 \equiv 1 \pmod{4}$, but $5 \not\equiv 1 \pmod{12}$, because $5 - 1 = 4$ is a multiple of 4 but not of 12. Like 1 a.m. and 1 p.m.: they match on a 12-hour clock but not on a 24-hour one. Going down to a divisor can only lose information.

WHY THIS MATTERS

Sometimes it is easier to work mod a big number N and switch to a smaller M afterwards. When M divides N the switch is always safe. The same idea comes up in the Chinese Remainder Theorem and in some primality tests later in the course.